

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск подстроки в строке

Студент гр. 4345

Трушкин А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм Кнут-Моррис-Пратт.

Постановка задачи

Часть 1. Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 500000$) найдите все вхождения P в T .

Ввод: строка P (шаблон), строка T (текст).

Вывод: индексы начал вхождений P в T , разделенные запятой; если P не входит в T , то вывести -1.

Ввод

ab

abab

Вывод

0, 2

Часть 2. Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$). Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход: первая строка A , вторая строка B

Выход: если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе -1. Если возможно несколько сдвигов, вывести первый индекс.

Ввод

defabc

abcdef

Вывод

3

Выполнение работы

Алгоритм Кнута-Морриса-Пратта (КМП) является эффективным методом поиска подстроки, позволяющим избежать лишних сравнений символов. В отличие от наивного метода, имеющего сложность $O(N \cdot M)$, где N - длина текста, а M - длина шаблона, КМП анализирует структуру шаблона с помощью префикс-функции.

1. Префикс функция. Процесс начинается с вычисления массива pi , где каждый элемент $pi[i]$ хранит длину самого длинного собственного префикса подстроки шаблона, совпадающего с её суффиксом. Основным циклом последовательно перебирает символы шаблона, увеличивая длину текущего совпадения k . В случае несовпадения символов срабатывает цикл `while`, который не сбрасывает поиск в ноль, а откатывает k к значению $pi[k-1]$. Это позволяет проверить наличие более коротких повторяющихся цепочек внутри уже обработанной части, исключая лишние проверки. Благодаря тому, что k увеличивается максимум на единицу за итерацию, суммарное количество откатов ограничено, что обеспечивает линейную сложность $O(M)$.

2. Алгоритм циклического сдвига. Задача поиска циклического сдвига была сведена к поиску вхождения строки A в строку B в условиях их циклической эквивалентности.

В процессе реализации использована оптимизация доступа к символам: вместо физического дублирования строки B , что потребовало бы $O(M)$ дополнительной памяти, используется оператор взятия остатка от деления: $A[i \% n]$. Это позволяет эмулировать "бесконечную" циклическую строку, обеспечивая поиск сдвига за один линейный проход.

Указатель q отслеживает глубину совпадения шаблона. При несовпадении символов q сдвигается согласно значениям массива pi , что позволяет избежать повторного сравнения уже проверенных символов. При достижении $q == m$ фиксируется индекс начала вхождения, соответствующий величине сдвига.

3. Сложность алгоритма

- Временная сложность: $O(N + M)$, где N - длина текста, M - длина шаблона. Построение префикс-функции занимает $O(M)$, а линейный проход по тексту с использованием откатов q - $O(N)$.

- Пространственная сложность: $O(M)$, так как для поиска необходимо хранить только строку-шаблон и массив префикс-функции. Использование операции $\%$ для эмуляции циклического сдвига позволяет избежать дополнительного выделения памяти под дубликат строки.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1-2.

Таблица 1 – Тестирование kmp_1

№	Входные данные	Выходные данные	Комментарий
1	ab abab	0,2	Базовый пример
2	aaa aaaaa	0,1,2	Базовый пример
3	abc def	-1	Случай полного отсутствия вхождений
4	a aaaaa	0,1,2,3,4	Шаблон из одного символа

Таблица 2 – Тестирование kmp_2

№	Входные данные	Выходные данные	Комментарий
1	defabc abcdef	3	Базовый пример
2	baaaba ababaa	3	Стандартный случай
3	abcd abcd	0	Строки идентичны
4	abc def	-1	Строки не являются сдвигами друг друга

Выводы

В ходе лабораторной работы был реализован алгоритм Кнута-Морриса-Пратта, обеспечивающий эффективный поиск подстроки и определение циклической эквивалентности строк за линейное время $O(N+M)$. Использование префикс-функции позволило исключить избыточные сравнения символов: алгоритм анализирует структуру шаблона и при несовпадении выполняет смещение указателя на безопасную дистанцию, предотвращая повторный просмотр уже изученных участков текста.

Для решения задачи о циклическом сдвиге была применена оптимизация с использованием оператора взятия остатка от деления, что позволило избежать физического дублирования данных и сократить потребление памяти до $O(M)$. Пошаговая система логирования обеспечила наглядность вычислений, позволяя отследить работу механизма откатов и процесс обновления состояния указателя шаблона в реальном времени. Тестирование подтвердило высокую производительность программы и её устойчивость к граничным случаям, таким как отсутствие вхождений или строки различной длины.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл kmp_1.py

```
def compute_pi(p):
    m = len(p)
    pi = [0] * m
    k = 0
    print(f"\nВычисление префикс-функции (pi) для шаблона '{p}':")
    for q in range(1, m):
        while k > 0 and p[q] != p[k]:
            k = pi[k-1]
        if p[q] == p[k]:
            k += 1
        pi[q] = k
    print(f"Массив pi: {pi}")
    return pi

def kmp_search(p, t):
    m, n = len(p), len(t)
    pi = compute_pi(p)
    res = []
    q = 0

    print(f"\nПоиск вхождений '{p}' в тексте '{t}':")
    for i in range(n):
        while q > 0 and p[q] != t[i]:
            print(f"Несовпадение: T[{i}]='{t[i]}' != P[{q}]='{p[q]}' , откат q на {pi[q-1]}")
            q = pi[q-1]

        if p[q] == t[i]:
            print(f"Совпадение: T[{i}]='{t[i]}' == P[{q}]='{p[q]}' , новый q={q+1}")
            q += 1

        if q == m:
            start_index = i - m + 1
```

```

        print(f"Найдено вхождение на позиции {start_index}")
        res.append(str(start_index))
        q = pi[q-1]

    return res

p = input("Введите шаблон (P): ")
t = input("Введите текст (T): ")
matches = kmp_search(p, t)

print("\nРезультат:")
if not matches:
    print("-1")
else:
    print(", ".join(matches))

```

Файл kmp_2.py

```

def get_pi(p):
    m = len(p)
    pi = [0] * m
    k = 0
    print(f"\nВычисляем префикс-функцию для шаблона '{p}'")
    for i in range(1, m):
        while k > 0 and p[i] != p[k]:
            k = pi[k-1]
        if p[i] == p[k]:
            k += 1
        pi[i] = k
    print(f"pi-массив = {pi}")
    return pi

def cyclic_shift(a, b):
    n, m = len(a), len(b)
    if n != m:
        print("-1")
        return
    if n == 0:
        print("0")
        return

```



```

pi = get_pi(b)
print(f"\nНачинаем поиск в тексте длиной {n} символов")
q = 0

for i in range(2 * n):
    char_a = a[i % n]
    print(f"Шаг {i}: Символ текста A[{i % n}] = '{char_a}',
сопоставляем с B[{q}] = '{b[q]}'")

    while q > 0 and char_a != b[q]:
        q = pi[q-1]
        print(f"Символы не совпали, откат по префикс
функции: q = {q}")

    if char_a == b[q]:
        q += 1
        print(f"Символы совпали идем вперед по префикс-
функции: q = {q}")

    if q == m:
        start_index = i - m + 1
        print(f"Найдено вхождение, Индекс начала:
{start_index}")

        print(start_index)
        return

print("-1")

str_a = input("Введите строку A: ").strip()
str_b = input("Введите строку B: ").strip()
cyclic_shift(str_a, str_b)

```